7️⃣

Buổi 7: YARP API Gateway

Giáo án Buổi 7: YARP API Gateway

0. Vị trí của buổi 7 trong lộ trình

Bạn đã hoàn thành checkpoint sau 6 buổi đầu:

```
Buổi 1: Minimal API + CQRS + MediatR
Buổi 2: Database + Repository + Dapper
Buổi 3: CatalogService + BasketService + Redis
Buổi 4: REST/Refit Service Communication
Buổi 5: gRPC Service Communication
Buổi 6: REST vs gRPC + Failure Mindset
```

Buổi 7 bắt đầu một phase mới:

```
API Gateway
```

Trước buổi này, client gọi trực tiếp từng service:

```
Client → CatalogService
Client → BasketService
```

Sau buổi này, client sẽ gọi qua Gateway:

```
Client → ApiGateway → CatalogService
                    → BasketService
```

---

1. Mục tiêu buổi học

Sau buổi này, bạn cần làm được:

1. Hiểu API Gateway là gì.
2. Hiểu Reverse Proxy là gì.
3. Hiểu YARP là gì trong .NET.
4. Tạo project `ApiGateway`.
5. Cấu hình route từ Gateway tới `CatalogService`.
6. Cấu hình route từ Gateway tới `BasketService`.
7. Gọi Catalog/Basket thông qua Gateway.
8. Hiểu Gateway là entry point của hệ thống.
9. Biết vì sao Gateway không nên chứa business logic nặng.
10. Biết các rủi ro nếu Gateway cấu hình sai.

---

2. Flow trước và sau Buổi 7

2.1. Flow trước Buổi 7

Client phải nhớ port từng service:

```
CatalogService: https://localhost:7079/products
BasketService : https://localhost:7081/basket/user-1
```

Vấn đề:

```
Client phải biết quá nhiều service phía sau.
Nếu service đổi port/base URL, client bị ảnh hưởng.
Sau này thêm auth/rate limit/logging sẽ bị lặp ở nhiều service.
```

2.2. Flow sau Buổi 7

Client chỉ gọi Gateway:

```
ApiGateway: https://localhost:7000
```

Gateway route request:

```
GET  /catalog/products            → CatalogService /products
POST /catalog/products            → CatalogService /products
GET  /basket/basket/user-1        → BasketService /basket/user-1
POST /basket/basket/user-1/items  → BasketService /basket/user-1/items
```

Flow:

```
Client
  ↓
ApiGateway
  ↓ route theo path
CatalogService / BasketService
```

---

3. Kiến thức nền

3.1. API Gateway là gì?

API Gateway là entry point đứng trước các backend services.

Thay vì client gọi trực tiếp từng service, client gọi Gateway. Gateway quyết định request nên chuyển đến service nào.

```
Client → Gateway → Service thật
```

Gateway thường làm các việc:

```
routing
reverse proxy
authentication/authorization
rate limiting
request logging
CORS
```

```
load balancing
header forwarding
```

Trong Buổi 7, mình chỉ học phần nền tảng nhất:

```
routing + reverse proxy
```

Các phần auth/rate limit sẽ học sau.

3.2. Reverse Proxy là gì?

Reverse Proxy là server đứng trước backend service và chuyển request vào service phía sau.

Ví dụ:

```
Client gọi: /catalog/products
Gateway chuyển tới: CatalogService /products
```

Client không cần biết CatalogService đang chạy ở port nào.

3.3. YARP là gì?

YARP là viết tắt của:

```
Yet Another Reverse Proxy
```

YARP là reverse proxy toolkit của .NET, dùng để xây API Gateway hoặc proxy service.

Trong project này:

```
ApiGateway dùng YARP để route request tới CatalogService và BasketService.
```

3.4. Route và Cluster trong YARP

YARP có 2 khái niệm quan trọng:

```
Route   : request nào sẽ được match?
Cluster : request đó sẽ được gửi tới service nào?
```

Ví dụ:

```
Route:
/catalog/{**catch-all}

Cluster:
CatalogService cluster → https://localhost:7079
```

Nghĩa là:

```
Request bắt đầu bằng /catalog
→ chuyển tới CatalogService
```

3.5. Path transform là gì?

Nếu client gọi Gateway:

```
/catalog/products
```

Nhưng CatalogService thật chỉ có endpoint:

```
/products
```

thì Gateway cần bỏ prefix `/catalog` trước khi forward.

Đó là path transform:

```
/catalog/products → /products
```

Trong YARP dùng:

```
"Transforms": [
  { "PathRemovePrefix": "/catalog" }
]
```

---

4. Thiết kế Gateway trong bài này

Ta sẽ tạo service mới:

```
ApiGateway
```

Các route mục tiêu:

Gateway Path	Forward tới service
`/catalog/{**catch-all}`	CatalogService
`/basket/{**catch-all}`	BasketService

Ví dụ:

```
GET https://localhost:7000/catalog/products
→ GET https://localhost:7079/products
```

```
GET https://localhost:7000/basket/basket/user-1
→ GET https://localhost:7081/basket/user-1
```

Lưu ý: vì BasketService endpoint hiện tại đã bắt đầu bằng `/basket`, nên khi qua Gateway path có thể thành `/basket/basket/...`. Sau này có thể thiết kế lại prefix đẹp hơn, nhưng buổi này ưu tiên hiểu YARP routing trước.

---

5. Tạo project ApiGateway

Tại thư mục solution:

```
dotnet new web -n ApiGateway
```

Thêm project vào solution nếu đang dùng `.sln`:

```
dotnet sln add ApiGateway/ApiGateway.csproj
```

Cài YARP:

```
dotnet add ApiGateway package Yarp.ReverseProxy
```

---

6. Cấu hình appsettings.json cho ApiGateway

Mở:

```
ApiGateway/appsettings.json
```

Nội dung mẫu:

```
{
  "ReverseProxy": {
    "Routes": {
      "catalog-route": {
        "ClusterId": "catalog-cluster",
        "Match": {
          "Path": "/catalog/{**catch-all}"
        },
        "Transforms": [
          { "PathRemovePrefix": "/catalog" }
        ]
      },
      "basket-route": {
```

```
        "ClusterId": "basket-cluster",
        "Match": {
          "Path": "/basket/{**catch-all}"
        },
        "Transforms": [
          { "PathRemovePrefix": "/basket" }
        ]
      }
    },
    "Clusters": {
      "catalog-cluster": {
        "Destinations": {
          "catalog-destination": {
            "Address": "https://localhost:7079/"
          }
        }
      },
      "basket-cluster": {
        "Destinations": {
          "basket-destination": {
            "Address": "https://localhost:7081/"
          }
        }
      }
    }
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Quan trọng:

```
Address phải là URL thật của service đang chạy trong Rider.
```

Nếu CatalogService của bạn là:

```
https://localhost:7079
```

thì để:

```
"Address": "https://localhost:7079/"
```

Nếu BasketService port khác, sửa đúng port.

---

7. Program.cs của ApiGateway

Mở:

```
ApiGateway/Program.cs
```

Sửa thành:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("Reverse
Proxy"));

var app = builder.Build();

app.MapGet("/", () => Results.Ok(new
{
    service = "ApiGateway",
    status = "Running"
}));
```

```
app.MapReverseProxy();

app.Run();
```

Giải thích:

```
AddReverseProxy() đăng ký YARP.
LoadFromConfig(...) đọc route/cluster từ appsettings.json.
MapReverseProxy() bật proxy routing.
```

---

8. Chạy các service

Bạn cần chạy ít nhất 3 project:

```
CatalogService
BasketService
ApiGateway
```

Nếu BasketService dùng Redis thì chạy thêm Redis.

```
docker start microshop-redis
```

Trong Rider, tạo Multi-Launch gồm:

```
CatalogService
BasketService
ApiGateway
```

Port ví dụ:

```
CatalogService: https://localhost:7079
BasketService : https://localhost:7081
ApiGateway    : https://localhost:7000
```

---

9. Test qua Gateway

9.1. Test Gateway sống

```
GET https://localhost:7000/
```

Kỳ vọng:

```
{
  "service": "ApiGateway",
  "status": "Running"
}
```

9.2. Test Catalog qua Gateway

Nếu CatalogService có endpoint:

```
GET /products
```

thì gọi qua Gateway:

```
GET https://localhost:7000/catalog/products
```

YARP sẽ chuyển thành:

```
GET https://localhost:7079/products
```

9.3. Test get product by id qua Gateway

```
GET https://localhost:7000/catalog/products/{productId}
```

9.4. Test Basket qua Gateway

Nếu BasketService có endpoint:

```
GET /basket/{userId}
```

Vì route Gateway là `/basket/{**catch-all}` và có remove prefix `/basket`, gọi:

```
GET https://localhost:7000/basket/basket/user-1
```

Gateway remove prefix đầu tiên:

```
/basket/basket/user-1 → /basket/user-1
```

rồi forward sang BasketService.

9.5. Test add basket item qua Gateway

```
POST https://localhost:7000/basket/basket/user-1/items
Content-Type: application/json

{
  "productId": "PASTE_PRODUCT_ID_HERE",
  "quantity": 2
}
```

---

10. Vấn đề path `/basket/basket` và cách thiết kế đẹp hơn

Hiện tại hơi xấu:

```
/basket/basket/user-1
```

Vì:

```
Gateway prefix: /basket
BasketService endpoint thật: /basket/{userId}
```

Có 2 hướng:

Hướng A: Giữ như hiện tại để học YARP trước

```
Gateway: /basket/basket/user-1
Service: /basket/user-1
```

Ưu điểm:

```
Không cần sửa BasketService.
Dễ hiểu PathRemovePrefix.
```

Hướng B: Thiết kế route Gateway đẹp hơn

Ví dụ Gateway dùng:

```
/cart/{**catch-all}
```

Cấu hình:

```
"basket-route": {
  "ClusterId": "basket-cluster",
  "Match": {
    "Path": "/cart/{**catch-all}"
  },
  "Transforms": [
    { "PathPattern": "/basket/{**catch-all}" }
  ]
}
```

Khi đó:

```
GET /cart/user-1
→ GET /basket/user-1
```

Buổi này ưu tiên Hướng A. Bài tập biến thể sẽ cho bạn làm Hướng B.

---

11. Lỗi hay gặp

Lỗi	Nguyên nhân	Cách xử lý
502 Bad Gateway	Service phía sau chưa chạy/sai port	Kiểm tra Catalog/Basket URL
404 từ Gateway	Path không match route	Kiểm tra `/catalog/...` hoặc `/basket/...`
404 từ service phía sau	Prefix chưa remove đúng	Kiểm tra `PathRemovePrefix`
SSL/certificate error	Gateway gọi service HTTPS dev cert lỗi	Trust dev cert, chạy cùng HTTPS local
Route đúng nhưng body lỗi	Request body không đúng DTO	Test trực tiếp service trước

---

12. Production mindset

API Gateway rất mạnh, nhưng không nên nhét quá nhiều business logic vào Gateway.

Gateway nên xử lý:

```
routing
authentication/authorization ở mức entry point
rate limiting
CORS
request logging
header forwarding
load balancing
```

Gateway không nên xử lý nặng như:

```
tính giá sản phẩm
quyết định nghiệp vụ checkout
lưu order
xử lý payment
business rule chi tiết của service
```

Lý do:

```
Nếu Gateway ôm business logic, nó dễ trở thành mini-monolith mới.
```

---

13. Thực hành chính

Bạn cần làm các bước:

```
[ ] Tạo project ApiGateway
[ ] Cài Yarp.ReverseProxy
[ ] Cấu hình route catalog-route
[ ] Cấu hình route basket-route
[ ] Đăng ký AddReverseProxy trong Program.cs
[ ] MapReverseProxy
[ ] Chạy CatalogService + BasketService + ApiGateway
[ ] Gọi Catalog qua Gateway
[ ] Gọi Basket qua Gateway
[ ] Test lỗi khi tắt CatalogService hoặc BasketService
```

---

14. Bài tập biến thể

> Không lặp lại phần thực hành chính. Các bài dưới đây giúp bạn hiểu sâu hơn Gateway routing.

14.1. Bài 1: Thiết kế route `/cart` đẹp hơn cho BasketService

Hiện tại:

```
/basket/basket/user-1
```

Hãy cấu hình Gateway để client gọi:

```
GET /cart/user-1
POST /cart/user-1/items
```

nhưng BasketService vẫn nhận:

```
GET /basket/user-1
POST /basket/user-1/items
```

Yêu cầu:

```
[ ] Tạo route mới `/cart/{**catch-all}`
[ ] Transform path về``/basket/{**catch-all}`
[ ] Test get basket qua `/cart/user-1`
[ ] Test add item qua `/cart/user-1/items`
```

14.2. Bài 2: Thêm response header từ Gateway

Thêm một header để biết response đi qua Gateway:

```
X-Gateway: YARP
```

Gợi ý transform trong YARP:

```
"Transforms": [
  { "ResponseHeader": "X-Gateway", "Set": "YARP" }
]
```

Yêu cầu:

```
[ ] Header xuất hiện khi gọi qua Gateway
[ ] Header không xuất hiện khi gọi trực tiếp service
```

14.3. Bài 3: Test 502 Bad Gateway

Tắt CatalogService rồi gọi:

```
GET /catalog/products
```

Yêu cầu trả lời:

```
1. Gateway trả lôĩ gì?
2. Vì sao lôĩ này xảy ra?
3. Khác gì với 404 Product not found?
4. Nêú là production, cân log gì?
```

14.4. Bài 4: Giải thích flow Gateway

Trả lời:

1. API Gateway là gì?
2. Reverse Proxy là gì?
3. YARP route là gì?
4. YARP cluster là gì?
5. `PathRemovePrefix` dùng để làm gì?
6. Vì sao client không nên gọi trực tiếp từng service khi hệ thống lớn dần?
7. Gateway nên xử lý những việc gì?
8. Gateway không nên xử lý những việc gì?
9. 502 khác 404 như thế nào?
10. Gateway có làm mất runtime coupling giữa service-to-service không?

---

15. Đáp án gợi ý ngắn

15.1. Route `/cart` cho BasketService

Ví dụ cấu hình route:

```
"cart-route": {
  "ClusterId": "basket-cluster",
  "Match": {
    "Path": "/cart/{**catch-all}"
  },
  "Transforms": [
    { "PathPattern": "/basket/{**catch-all}" }
  ]
}
```

Khi gọi:

```
/cart/user-1
```

Gateway forward:

```
/basket/user-1
```

15.2. 502 vs 404

```
502 Bad Gateway:
Gateway không forward được tới service phía sau hoặc service phía sau unavailable.

404 Not Found:
Route/resource không tồn tại. Có thể là Gateway không match route, hoặc service phía sau
không tìm thấy resource.
```

Ví dụ:

```
CatalogService down → Gateway có thể trả 502.
Product id không tồn tại nhưng CatalogService vẫn chạy → CatalogService trả 404.
```

---

16. Checklist pass Buổi 7

```
[ ] Hiểu API Gateway là gì
[ ] Hiểu Reverse Proxy là gì
[ ] Hiểu YARP là gì
[ ] Tạo được ApiGateway project
[ ] Cài được Yarp.ReverseProxy
[ ] Câú hình được catalog-route
[ ] Câú hình được basket-route
[ ] Gọi được CatalogService qua Gateway
[ ] Gọi được BasketService qua Gateway
[ ] Hiểu PathRemovePrefix
[ ] Biết phân biệt 502 và 404
[ ] Làm ít nhất 2 bài tập biến thể
[ ] Trả lời được câu hỏi flow Gateway
```

---

17. Điều kiện mở khóa Buổi 8

Bạn sẽ sang Buổi 8 khi:

```
Client gọi được Catalog/Basket qua ApiGateway.
Bạn hiểu Gateway là entry point.
Bạn biết Gateway route request chứ không nên ôm business logic nặng.
Bạn phân biệt được lỗi route/service unavailable/resource not found.
```

Buổi 8 sẽ học:

```
Docker Compose cơ bản
```

Mục tiêu Buổi 8:

```
Chạy nhiều service bằng một lệnh.
Chuẩn bị hệ thống local giống môi trường thật hơn.
```